

UNIVERSITE DE NATITINGOU
FACULTE DES SCIENCES ET TECHNIQUES (FAST)
Departement de Mathematiques et Informatique
Laboratoire IA CAEB / Fondation VALLET

RAPPORT DE STAGE

Implementation *from scratch* d'un reseau de neurones multicouche applique a la classification

*Comparaison avec scikit-learn — Extension aux images (MNIST,
CIFAR-10, CNN)*

Etudiant	Frederic BALAGUEMA
Filiere	Mathematiques Informatique (MI3)
Etablissement	FAST Natitingou
Laboratoire	IA CAEB / Fondation VALLET
Contact	fredericbalaguema@gmail.com
GitHub	https://github.com/Fredericbalaguema-Ops/reseau_neurones_from_scrat
Colab	https://colab.research.google.com/drive/1WU8zWtLvovyLx6jfZ07kj4AhrA
Kaggle	https://www.kaggle.com/code/fredericbalaguema/reseau-neurones-from-
Annee	2025 – 2026

Resume

Ce rapport presente les travaux realises dans le cadre du **Theme 9** du stage de fin de formation en Mathematiques Informatique (MI3) a la FAST de Natitingou.

L'objectif principal est l'implementation *from scratch*, en Python avec uniquement NumPy, d'un reseau de neurones multicouche (MLP). Cette implementation couvre l'ensemble du pipeline : propagation avant, retropropagation du gradient, et descente de gradient stochastique (SGD).

Le reseau est applique a trois problemes de complexite croissante : Iris (donnees tabulaires), MNIST (images de chiffres) et CIFAR-10 (photos couleur). Une comparaison avec `MLPClassifier` de scikit-learn valide l'implementation. Sur demande du maitre de stage, le projet est etendu aux reseaux convolutifs (CNN) avec TensorFlow/Keras.

Resultats obtenus :

- MLP from scratch sur Iris : **96.7%** accuracy TEST (identique a scikit-learn)
- MLP from scratch sur MNIST : **91.9%** accuracy TEST
- MLP from scratch sur CIFAR-10 : **33.8%** accuracy TEST (limite du MLP sur photos couleur)
- CNN Keras sur CIFAR-10 : **67.7%** accuracy TEST (+33.9 points vs MLP)
- Gradient check valide : difference relative 6.57×10^{-11}

Mots-cles : reseau de neurones, MLP, backpropagation, NumPy, from scratch, classification, MNIST, CIFAR-10, CNN, Keras.

Table des matières

Resume	1
1 Introduction	3
1.1 Contexte du stage	3
1.2 Problematique	3
1.3 Objectifs	3
2 Fondements Theoriques	5
2.1 Architecture du MLP	5
2.2 Fonctions d'activation	5
2.2.1 ReLU — couches cachees	5
2.2.2 Sigmoides — sortie binaire	5
2.2.3 Softmax — sortie multi-classes	6
2.3 Fonction de cout	6
2.4 Retropropagation du gradient	6
2.5 Descente de gradient stochastique (SGD)	7
3 Implementation From Scratch	8
3.1 Environnement technique	8
3.2 Fonctions implementees	8
3.3 Initialisation des poids	9
3.4 Validation par Gradient Check	9
4 Resultats sur Iris et Comparaison avec scikit-learn	10
4.1 Dataset Iris	10
4.2 Architecture et hyperparametres	10
4.3 Evolution de l'entrainement	11
4.4 Comparaison avec scikit-learn	11
4.5 Analyse des hyperparametres	11
4.5.1 Impact du learning rate	11

5	Application aux Images	12
5.1	Representation numerique d'une image	12
5.2	MNIST — Chiffres manuscrits	12
5.2.1	Description	12
5.2.2	Resultats	13
5.3	CIFAR-10 — Photos couleur	13
5.3.1	Description	13
5.3.2	Resultats MLP sur CIFAR-10	13
6	Extension : Introduction au CNN	15
6.1	Motivation	15
6.2	Principe de la convolution	15
6.3	Architecture du CNN utilise	16
6.4	Resultats et comparaison finale	16
7	Conclusion et Perspectives	18
7.1	Bilan	18
7.2	Competences acquises	18
7.3	Perspectives	18
	Bibliographie	20
	Annexes	21

Chapitre 1

Introduction

1.1 Contexte du stage

Ce stage s'inscrit dans la formation en Mathématiques Informatique (MI3) à la FAST de Natitingou, au sein du laboratoire IA CAEB / Fondation VALLET (année 2025-2026).

L'intelligence artificielle et le machine learning connaissent un essor considérable. En Afrique et au Bénin particulièrement, la maîtrise de ces technologies représente un enjeu stratégique. Ce stage adopte une approche pédagogique rigoureuse : comprendre les algorithmes non pas en utilisant des boîtes noires, mais en les implementant de A à Z.

1.2 Problematique

La majorité des formations ML utilisent des bibliothèques de haut niveau (TensorFlow, PyTorch, scikit-learn) qui masquent les mécanismes fondamentaux. Un ingénieur qui ne comprend pas ce qui se passe derrière `model.fit()` est limité dans sa capacité à diagnostiquer, optimiser et innover.

Question centrale : est-il possible d'implementer un réseau de neurones multicouche performant en utilisant uniquement NumPy, et d'atteindre des performances comparables à des frameworks optimisés ?

1.3 Objectifs

1. Implementer from scratch en NumPy : propagation avant, retropropagation, SGD, fonctions d'activation.
2. Appliquer le réseau à des problèmes concrets de classification (Iris, MNIST, CIFAR-10).
3. Comparer les performances avec scikit-learn.

4. Analyser l'impact des hyperparametres.
5. Etendre le projet aux images et introduire le CNN.

Chapitre 2

Fondements Theoriques

2.1 Architecture du MLP

Un MLP (Multilayer Perceptron) est organise en couches successives. Pour chaque couche l , le calcul comporte deux etapes :

Formule

Partie lineaire :

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

Activation :

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

ou $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$, $b^{[l]} \in \mathbb{R}^{n_l \times 1}$, et $g^{[l]}$ est la fonction d'activation.

2.2 Fonctions d'activation

2.2.1 ReLU — couches cachees

Formule

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \mathbf{1}_{z>0}$$

2.2.2 Sigmoides — sortie binaire

Formule

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

2.2.3 Softmax — sortie multi-classes

Formule

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_{i=1}^K \text{softmax}(z_i) = 1$$

2.3 Fonction de cout

Pour la classification multi-classes, on utilise la cross-entropie :

Formule

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)})$$

2.4 Retropropagation du gradient

La backpropagation calcule les gradients couche par couche en remontant via la regle de derivation en chaine. Pour un reseau avec softmax + cross-entropy en sortie et ReLU dans les couches cachees :

Formule

$$\begin{aligned} dZ^{[L]} &= A^{[L]} - Y \quad (\text{gradient simplifie softmax + cross-entropy}) \\ dW^{[l]} &= \frac{1}{m} dZ^{[l]} (A^{[l-1]})^\top \\ db^{[l]} &= \frac{1}{m} \sum_{i=1}^m dZ^{[l]} \\ dA^{[l-1]} &= (W^{[l]})^\top dZ^{[l]} \\ dZ^{[l-1]} &= dA^{[l-1]} \times \text{ReLU}'(Z^{[l-1]}) \end{aligned}$$

2.5 Descente de gradient stochastique (SGD)

Formule

$$W^{[l]} \leftarrow W^{[l]} - \alpha \cdot dW^{[l]}, \quad b^{[l]} \leftarrow b^{[l]} - \alpha \cdot db^{[l]}$$

ou α est le taux d'apprentissage (*learning rate*).

Chapitre 3

Implementation From Scratch

3.1 Environnement technique

Outil	Version / Details
Langage	Python 3.10.11
Calcul	NumPy (exclusivement pour le MLP)
Visualisation	Matplotlib, Seaborn
Environnement	Jupyter Notebook + VS Code
Versionning	Git / GitHub (fredericbalaguema-ops)
CNN (extension)	TensorFlow / Keras

3.2 Fonctions implementees

Fonction	Role
sigmoid, relu, softmax	Fonctions d'activation
sigmoid_derivative, relu_derivative	Derivees pour la backprop
initialize_parameters	Initialisation Xavier/He
forward_propagation	Calcul de la prediction + stockage du cache
compute_loss	Cross-entropy binaire et multi-classes
backward_propagation	Calcul des gradients (backprop)
update_parameters	Mise a jour SGD
predict, compute_accuracy	Evaluation des performances
model	Boucle d'entrainement complete

3.3 Initialisation des poids

L'initialisation Xavier evite l'explosion/disparition du gradient :

Formule

$$W^{[l]} \sim \mathcal{N}\left(0, \frac{2}{n_{l-1}}\right)$$

Remarque

Si tous les poids sont initialises a zero, tous les neurones d'une couche produisent la meme sortie et apprennent la meme chose (probleme de symetrie). L'initialisation aleatoire brise cette symetrie.

3.4 Validation par Gradient Check

La correction de la backpropagation a ete validee par gradient check : comparaison du gradient analytique (backprop) avec le gradient numerique par differences finies.

Formule

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} \approx \frac{\mathcal{L}(W_{ij} + \varepsilon) - \mathcal{L}(W_{ij} - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-5}$$

Resultat

Gradient backprop $dW_1[0, 0]$	0.07094040
Gradient numerique $dW_1[0, 0]$	0.07094040
Difference relative	6.57×10^{-11}

La difference est quasi-nulle : la backpropagation est **mathematiquement exacte**.

Chapitre 4

Resultats sur Iris et Comparaison avec scikit-learn

4.1 Dataset Iris

Caracteristique	Valeur
Exemples totaux	150
Features	4 (longueur/largeur sepale et petale)
Classes	3 (Setosa, Versicolor, Virginica)
Split train/test	120 / 30 (80% / 20%)
Normalisation	StandardScaler

4.2 Architecture et hyperparametres

Hyperparametre	Valeur
Architecture	[4, 16, 8, 3]
Learning rate	0.05
Iterations	2000
Activation couches cachees	ReLU
Activation sortie	Softmax
Initialisation	Xavier/He
Nb de parametres	243

4.3 Evolution de l'entraînement

Iteration	Loss	Acc. TRAIN	Acc. TEST
0	2.0922	34.2%	23.3%
500	0.0544	98.3%	93.3%
1000	0.0353	98.3%	93.3%
1500	0.0286	98.3%	96.7%
1999	0.0246	98.3%	96.7%

Observation : le reseau démarre a 34.2% (proche du hasard sur 3 classes = 33.3%) et converge vers 96.7% en test des l'iteration 1500.

4.4 Comparaison avec scikit-learn

Resultat

Modele	Acc. TRAIN	Acc. TEST	Loss	Temps
MLP from scratch	98.3%	96.7%	0.0246	3.28 s
scikit-learn	100.0%	96.7%	0.0007	4.36 s

Les deux modeles atteignent la meme accuracy TEST (96.7%). Le MLP from scratch est meme 1.3x plus rapide que sklearn sur ce dataset.

4.5 Analyse des hyperparametres

4.5.1 Impact du learning rate

Learning rate	Loss finale	Acc. TEST
0.001	0.3309	76.7%
0.010	0.0751	93.3%
0.050	0.0256	96.7%
0.100	0.0137	96.7%
0.500	0.0005	93.3%

Conclusion : lr = 0.05 est optimal. Un lr trop petit ralentit la convergence (76.7% a 0.001), un lr trop grand cree de l'instabilite (93.3% a 0.5).

Chapitre 5

Application aux Images

5.1 Représentation numérique d’une image

Une image est une matrice de pixels, chaque pixel ayant une valeur entre 0 (noir) et 255 (blanc). Le pipeline de preprocessing pour notre MLP comporte 4 étapes :

1. **Normalisation** : $X_{\text{norm}} = X_{\text{raw}}/255.0$
2. **Aplatissement** : $(H \times W) \rightarrow (H \cdot W, 1)$
3. **Transposition** : $(m, n_x) \rightarrow (n_x, m)$
4. **One-hot encoding** : labels entiers $\rightarrow$ matrice binaire (K, m)

5.2 MNIST — Chiffres manuscrits

5.2.1 Description

Caractéristique	Valeur
Images totales	70 000
Train / Test	10 000 / 10 000
Taille image	28×28 pixels
Pixels par image	784
Type	Niveaux de gris
Classes	10 (chiffres 0–9)
Architecture	[784, 128, 64, 10]
Nb de paramètres	109 386

5.2.2 Resultats

Modele	Acc. TRAIN	Acc. TEST	Loss	Temps
MLP from scratch	93.4%	91.9%	0.2372	137.4 s
scikit-learn	100.0%	95.9%	—	140.9 s

Observation : le MLP from scratch atteint 91.9% sur MNIST en 300 iterations, proche des 95.9% de sklearn. La difference s'explique par les optimisations internes de sklearn (Adam, batch adaptatif).

5.3 CIFAR-10 — Photos couleur

5.3.1 Description

Caracteristique	Valeur
Images totales	60 000
Train / Test utilise	10 000 / 10 000
Taille image	32×32 pixels
Canaux	3 (RGB — couleur)
Pixels par image	$32 \times 32 \times 3 = 3072$
Classes	10 (avion, voiture, oiseau, chat...)
Architecture	[3072, 256, 128, 10]

5.3.2 Resultats MLP sur CIFAR-10

Modele	Acc. TRAIN	Acc. TEST	Temps
MLP from scratch	34.8%	33.8%	444.7 s
scikit-learn	100.0%	44.7%	—

Le MLP n'atteint que 33.8% sur CIFAR-10 (contre 10% pour le hasard). Cette limite s'explique par trois raisons :

1. **Perte de la structure spatiale :** l'aplatissement transforme la matrice 2D en vecteur 1D — le reseau ne sait plus que les pixels voisins sont lies.
2. **Variabilite intra-classe :** un meme objet peut apparaitre a differentes positions, angles et tailles.
3. **Complexite insuffisante :** capturer la complexite des photos couleur

necessite une architecture plus adaptee.

Chapitre 6

Extension : Introduction au CNN

6.1 Motivation

Les limitations du MLP sur CIFAR-10 ont motivé l'extension du projet aux réseaux de neurones convolutifs (CNN), conçus spécifiquement pour traiter des images en préservant leur structure spatiale.

6.2 Principe de la convolution

Un CNN applique des **filtres** qui glissent sur l'image. Chaque filtre détecte un type de pattern spécifique (bords, textures, formes). La convolution en position (i, j) pour un filtre K de taille $f \times f$ est :

Formule

$$(X \star K)[i, j] = \sum_{m=0}^{f-1} \sum_{n=0}^{f-1} X[i + m, j + n] \cdot K[m, n]$$

Les filtres sont **partagés** sur toute l'image, ce qui confère au CNN une invariance aux translations.

6.3 Architecture du CNN utilise

Couche	Output	Role
Conv2D(32, 3x3)+ReLU	$32 \times 32 \times 32$	Detection des contours
Conv2D(32, 3x3)+ReLU	$32 \times 32 \times 32$	Affinement
MaxPooling(2x2)	$16 \times 16 \times 32$	Reduction resolution
Dropout(0.25)	–	Regularisation
Conv2D(64, 3x3)+ReLU	$16 \times 16 \times 64$	Formes complexes
Conv2D(64, 3x3)+ReLU	$16 \times 16 \times 64$	Affinement
MaxPooling(2x2)	$8 \times 8 \times 64$	Reduction
Dropout(0.25)	–	Regularisation
Flatten	4096	Passage Dense
Dense(512)+ReLU	512	Classification
Dropout(0.5)	–	Regularisation forte
Dense(10)+Softmax	10	Probabilites

Optimiseur : Adam (lr=0.001) — plus efficace que SGD car il adapte le learning rate.

6.4 Resultats et comparaison finale

Iteration	Accuracy TRAIN	Accuracy TEST
Epoch 1	35.5%	16.6%
Epoch 5	65.7%	61.0%
Epoch 10	78.3%	64.0%
Epoch 13	83.1%	69.1%
Epoch 15	86.5%	67.7%

Resultat

Modele	Dataset	Acc. TEST	Temps
MLP from scratch	Iris	96.7%	3.3 s
MLP from scratch	MNIST	91.9%	137.4 s
MLP from scratch	CIFAR-10	33.8%	444.7 s
scikit-learn MLP	CIFAR-10	44.7%	–
CNN Keras	CIFAR-10	67.7%	1620 s

Le CNN obtient **+33.9 points** d'accuracy par rapport au MLP from scratch

sur CIFAR-10, grace a la preservation de la structure spatiale par les filtres convolutifs.

Chapitre 7

Conclusion et Perspectives

7.1 Bilan

Ce stage a permis d'atteindre tous les objectifs fixes :

- **Implementation complete et validee** : toutes les briques du MLP ont ete implementees en NumPy, avec validation mathematique par gradient check ($\delta = 6.57 \times 10^{-11}$).
- **Performances comparables a scikit-learn** : 96.7% sur Iris (identique), 91.9% vs 95.9% sur MNIST.
- **Limites identifiees et expliquees** : 33.8% sur CIFAR-10, justifiant l'existence des CNN.
- **Extension CNN reussie** : 67.7% avec CNN Keras, soit +33.9 points vs MLP.

7.2 Competences acquises

Mathematiques : derivees partielles, chain rule, algebre lineaire pour le ML.

Programmation : implementation vectorisee NumPy, gestion des shapes, debugging matriciel.

Machine Learning : comprehension profonde des mecanismes d'apprentissage, analyse des hyperparametres, detection d'overfitting.

Deep Learning : introduction au CNN, convolution, pooling, Keras.

7.3 Perspectives

1. Implementer le mini-batch SGD pour acclerer l'entrainement sur grands datasets.
2. Implementer l'optimiseur Adam from scratch.
3. Appliquer le reseau a un probleme concret du contexte beninois (agriculture, sante).

4. Approfondir le CNN : implementer la convolution from scratch en NumPy.

Bibliographie

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
<https://www.deeplearningbook.org>
2. Nielsen, M. (2015). *Neural Networks and Deep Learning*. <http://neuralnetworksanddeeplearning.com>
3. Machine Learnia. (2021). *Formation Deep Learning* [YouTube]. <https://youtu.be/XUFLq6dKQok>
4. 3Blue1Brown. (2017). *Neural Networks* [YouTube]. https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
5. Karpathy, A. (2022). *micrograd : backpropagation from scratch* [YouTube]. <https://youtu.be/VMj-3S1tku0>
6. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. University of Toronto. (Dataset CIFAR-10)

Annexes

Annexe A — Tableau des dimensions du reseau

Variable	Shape	Signification
X	(n_x, m)	Donnees d'entree
$W^{[l]}$	(n_l, n_{l-1})	Poids couche l
$b^{[l]}$	$(n_l, 1)$	Biais couche l
$Z^{[l]}, A^{[l]}$	(n_l, m)	Pre-act. et activation
$dW^{[l]}$	(n_l, n_{l-1})	Gradient — meme shape que W
$db^{[l]}$	$(n_l, 1)$	Gradient — meme shape que b

Annexe B — Comparaison complete des experiences

Experience	Architecture	Params	Acc. TEST	Framework
MLP Iris	[4, 16, 8, 3]	243	96.7%	NumPy
sklearn Iris	[4, 16, 8, 3]	243	96.7%	sklearn
MLP MNIST	[784, 128, 64, 10]	109 386	91.9%	NumPy
sklearn MNIST	[784, 128, 64, 10]	109 386	95.9%	sklearn
MLP CIFAR-10	[3072, 256, 128, 10]	821 514	33.8%	NumPy
sklearn CIFAR-10	[3072, 256, 128, 10]	821 514	44.7%	sklearn
CNN CIFAR-10	Conv+Dense	~470k	67.7%	Keras

Annexe C — Ressources en ligne

Plateforme	Lien
GitHub	https://github.com/Fredericbalaguema-Ops/reseau_neurones_from_scratch.ipynb
Google Colab	https://colab.research.google.com/drive/1WU8zWtLvovyLx6jfZ07kj4AhrAPMRNMG?usp=sharing
Kaggle	https://www.kaggle.com/code/fredericbalaguema/reseau-neurones-from-scratch

Structure du depot GitHub :

```

Fredericbalaguema-Ops/reseau_neurones_from_scratch.ipynb/
  README.md                    <- description du projet
  reseau_neurones_from_scratch.ipynb    <- notebook principal
  requirements.txt              <- dependances Python
  LICENSE
  figures/
    courbe_loss_accuracy.png
    comparaison_from_scratch_vs_sklearn.png
    analyse_learning_rate.png
    analyse_architecture.png
    underfitting_overfitting.png
    mnist_exemples.png
    mnist_resultats.png
    mnist_erreurs.png
    cifar10_exemples.png
    cnn_vs_mlp_cifar10.png
    cnn_predictions.png
  rapport/
    rapport_theme9_BALAGUEMA.pdf

```